

LAPORAN PRAKTIKUM PEMROGRAMAN WEB

MODUL 6: PEMROGRAMAN BERORIENTASI

OBJEK (OOP) PADA PHP



**Universitas
Telkom**

Disusun Oleh:

Nama: Andhika Yuridho

NIM: 707012500059

Kelas: D4SIKC-49-04

Mata Kuliah: Pemrograman Web

UNIVERSITAS TELKOM

FAKULTAS ILMU TERAPAN

JURUSAN S1 TERAPAN SISTEM INFORMASI KOTA Cerdas

TAHUN AJARAN 2026

A. Tujuan

Setelah mengikuti praktikum ini mahasiswa diharapkan dapat:

1. Memahami konsep dasar Pemrograman Berorientasi Objek (OOP) dalam PHP.
2. Menggunakan tipe data object dalam PHP.
3. Membuat dan menggunakan class serta object.
4. Memahami konsep constructor dan destructor.
5. Menerapkan konsep inheritance dalam OOP PHP.

B. Alat dan Bahan

Untuk menjalankan modul ini, diperlukan:

1. Komputer dengan sistem operasi Windows, macOS, atau Linux.
2. Web server seperti XAMPP.
3. Editor teks atau IDE seperti VS Code, Sublime Text, atau PHPStorm.
4. Browser web (Google Chrome, Mozilla Firefox, atau lainnya).

C. Langkah-Langkah Praktikum

a. Soal No 1

1. Perancangan Arsitektur dan Struktur Proyek

Langkah awal adalah memisahkan logika bisnis dari tampilan dengan membuat struktur direktori yang terorganisir:

- **src/**: Folder khusus untuk menyimpan semua file kelas (Class).
- **config/**: Folder untuk konfigurasi global dan inisialisasi sistem.

2. Implementasi Koneksi Database (Singleton Pattern)

Dibuat kelas **Database.php** menggunakan pola desain *Singleton*. Hal ini bertujuan untuk memastikan hanya ada satu koneksi database yang aktif selama satu siklus request, sehingga menghemat sumber daya server.

3. Pembuatan Base Model (Abstraksi CRUD)

Dibuat sebuah kelas induk abstrak bernama **BaseModel.php**. Kelas ini berfungsi sebagai kerangka kerja utama untuk operasi database (Create, Read, Update, Delete). Dengan adanya Base Model, kita tidak perlu menulis ulang kueri SQL yang sama di setiap file, cukup memanggil metode seperti `all()`, `find()`, `create()`, dan `update()`.

4. Pembuatan Kelas Entitas dan Logika Bisnis

Setelah kerangka dasar selesai, dibuat kelas-kelas spesifik yang mewakili entitas dan fungsi sistem:

- **User.php:** Mengelola data pengguna dan proses registrasi.
- **Doctor.php:** Mengelola data spesifik pada tabel dokter.
- **Auth.php:** Berisi logika statis untuk manajemen sesi (*session management*), login, logout, dan perlindungan halaman (*access control*).
- **FileHelper.php:** Berisi logika statis untuk menangani upload, validasi format/ukuran file, dan penghapusan file sertifikat di server.

5. Sistem Autoloading dan Inisialisasi Global

Dibuat file **config/init.php** yang mengimplementasikan `spl_autoload_register`. Sistem ini secara otomatis memuat file kelas dari folder `src/` saat sebuah objek dipanggil (contoh: memanggil `new User()` akan otomatis mencari file `src/User.php`). Ini menghilangkan kebutuhan akan banyak perintah `include` manual.

6. Refaktorisasi Halaman Program (Entry Points)

Mengubah semua halaman utama agar tidak lagi menggunakan fungsi prosedural, melainkan memanggil metode dari kelas yang telah dibuat.

- Contoh: Fungsi `getData()` pada `admin.php` diganti dengan `$doctorModel->all()`.
- Contoh: Logika login manual pada `login.php` diganti dengan `Auth::login()`.

7. Dokumentasi Kode dan Komentar Alur

Langkah terakhir adalah menambahkan komentar penjelasan (*Code Documentation*) pada setiap kelas dan alur logika penting. Ini dilakukan untuk memastikan kode memenuhi standar industri yang mudah dipahami (*readable*) dan dipelihara (*maintainable*).

8. Pembersihan (Cleanup)

Menghapus file-file prosedural lama yang sudah tidak digunakan (`db.php`, `functions.php`, dan `sessionCheck.php`) untuk memastikan proyek bersih dan tidak ada redundansi kode.

D. Hasil dan Dokumentasi Program

a. Soal No 1

config/init.php

```
<?php

/**
 * File Inisialisasi Global
 * Mengatur sesi dan sistem autoloading kelas.
 */

// Mulai sesi secara global
if (session_status() === PHP_SESSION_NONE) {
    session_start();
}

/**
 * Autoloader Kelas
 * Secara otomatis memuat file kelas dari folder 'src/' saat sebuah
 * kelas dipanggil.
 * Contoh: Memanggil 'new User()' akan memuat 'src/User.php'.
 */
spl_autoload_register(function ($className) {
    $path = __DIR__ . '/../src/' . $className . '.php';
    if (file_exists($path)) {
        require_once $path;
    }
});
```

src/Auth.php

```
<?php

/**
 * Class Auth
 * Mengelola autentikasi user dan manajemen sesi.
 */
class Auth {
    /**
     * Memulai sesi jika belum dimulai.
     */
    public static function startSession() {
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }
    }
}
```

```

/**
 * Proses login user.
 * @return bool True jika login berhasil, false jika gagal.
 */
public static function login($email, $password) {
    $userModel = new User();
    $user = $userModel->findByEmail($email);

    if ($user && password_verify($password, $user['password'])) {
        self::startSession();
        $_SESSION["isLogin"] = true;
        $_SESSION["username"] = $user["name"];
        return true;
    }
    return false;
}

/**
 * Proses logout user.
 */
public static function logout() {
    self::startSession();
    session_destroy();
    header("Location: login.php");
    exit();
}

/**
 * Memastikan user sudah login. Jika belum, redirect ke halaman
login.
 */
public static function checkLogin() {
    self::startSession();
    if (!isset($_SESSION['isLogin']) || !$_SESSION['isLogin']) {
        header("Location: login.php");
        exit();
    }
}

/**
 * Redirect user ke dashboard jika sudah login (digunakan di
halaman login/register).
 */
public static function redirectIfLoggedIn() {
    self::startSession();
    if (isset($_SESSION['isLogin']) && $_SESSION['isLogin']) {
        header("Location: admin.php");
    }
}

```

```

        exit();
    }
}
}

```

src/BaseModel.php

```

<?php

/**
 * Class BaseModel
 * Menyediakan fungsionalitas CRUD dasar untuk semua model di aplikasi.
 */
abstract class BaseModel {
    protected $db;
    protected $table;
    protected $primaryKey = 'id';

    /**
     * Inisialisasi koneksi database saat objek dibuat.
     */
    public function __construct() {
        $this->db = Database::getInstance()->getConnection();
    }

    /**
     * Menentukan tipe data untuk binding parameter MySQLi (integer,
     double, atau string).
     */
    protected function getBindType($value) {
        if (is_int($value)) return 'i';
        if (is_float($value) || is_double($value)) return 'd';
        return 's';
    }

    /**
     * Mengambil semua data dari tabel, mendukung filter conditions.
     * @param array $conditions Array asosiatif ['kolom' => 'nilai']
     */
    public function all($conditions = []) {
        $sql = "SELECT * FROM {$this->table}";
        $types = '';
        $values = [];

        if (!empty($conditions)) {
            $clauses = [];
            foreach ($conditions as $key => $value) {

```

```

        $cleanKey = trim($key);
        if (strpos($cleanKey, ' ') !== false) {
            // Mendukung operator custom (misal: 'id >')
            $clauses[] = "$cleanKey ?";
        } else {
            $clauses[] = "$cleanKey = ?";
        }
        $types .= $this->getBindType($value);
        $values[] = $value;
    }
    $sql .= " WHERE " . implode(' AND ', $clauses);
}

$stmt = $this->db->prepare($sql);
if (!empty($values)) {
    $stmt->bind_param($types, ...$values);
}
$stmt->execute();
return $stmt->get_result()->fetch_all(MYSQLI_ASSOC);
}

/**
 * Mencari satu record berdasarkan ID (Primary Key).
 */
public function find($id) {
    $data = $this->all([$this->primaryKey => $id]);
    return (!empty($data)) ? $data[0] : null;
}

/**
 * Menyisipkan record baru ke tabel secara dinamis.
 * @param array $data Array asosiatif ['kolom' => 'nilai']
 */
public function create($data) {
    $columns = implode(', ', array_keys($data));
    $placeholders = implode(', ', array_fill(0, count($data),
'?'));
    $values = array_values($data);

    $types = '';
    foreach ($values as $value) {
        $types .= $this->getBindType($value);
    }

    $sql = "INSERT INTO {$this->table} ($columns) VALUES
($placeholders)";
    $stmt = $this->db->prepare($sql);
    $stmt->bind_param($types, ...$values);

```

```

        return $stmt->execute();
    }

    /**
     * Memperbarui record berdasarkan ID secara dinamis.
     * @param mixed $id Nilai primary key
     * @param array $data Data baru yang akan diupdate
     */
    public function update($id, $data) {
        $cols = [];
        $types = '';
        $values = [];

        foreach ($data as $key => $value) {
            $cols[] = "$key = ?";
            $types .= $this->getBindType($value);
            $values[] = $value;
        }

        $types .= $this->getBindType($id);
        $values[] = $id;

        $setClause = implode(', ', $cols);
        $sql = "UPDATE {$this->table} SET $setClause WHERE {$this->primaryKey} = ?";
        $stmt = $this->db->prepare($sql);
        $stmt->bind_param($types, ...$values);
        return $stmt->execute();
    }

    /**
     * Menghapus record berdasarkan ID.
     */
    public function delete($id) {
        $sql = "DELETE FROM {$this->table} WHERE {$this->primaryKey} = ?";
        $stmt = $this->db->prepare($sql);
        $stmt->bind_param($this->getBindType($id), $id);
        return $stmt->execute();
    }
}

```

src/Database.php

```
<?php
```

```
/**
```

```

* Class Database
* Mengelola koneksi database menggunakan pola Singleton untuk
memastikan
* hanya ada satu koneksi yang aktif selama siklus hidup request.
*/
class Database {
    private static $instance = null;
    private $conn;

    // Konfigurasi database
    private $host = "localhost";
    private $username = "root";
    private $password = "";
    private $db_name = "nuids_db";

    /**
     * Constructor private untuk mencegah instansiasi langsung dari
    luar kelas.
    */
    private function __construct() {
        $this->conn = new mysqli($this->host, $this->username, $this->password, $this->db_name);

        // Cek koneksi
        if ($this->conn->connect_error) {
            die("Koneksi Gagal! " . $this->conn->connect_error);
        }
    }

    /**
     * Mendapatkan instansi tunggal dari kelas Database.
    */
    public static function getInstance() {
        if (!self::$instance) {
            self::$instance = new Database();
        }
        return self::$instance;
    }

    /**
     * Mendapatkan objek koneksi mysqli.
    */
    public function getConnection() {
        return $this->conn;
    }
}

```

src/Doctor.php

```
<?php

/**
 * Class Doctor
 * Model untuk mengelola data pada tabel 'dokter'.
 */
class Doctor extends BaseModel {
    protected $table = 'dokter';
}
```

src/FileHelper.php

```
<?php

/**
 * Class FileHelper
 * Utilitas untuk mengelola operasi file (upload dan delete).
 */
class FileHelper {
    private static $uploadPath = 'uploads/';
    private static $allowedExtensions = ['jpg', 'jpeg', 'png', 'pdf', 'docx'];

    /**
     * Menangani upload file tunggal ke server.
     * @return array Status dan nama file atau pesan error.
     */
    public static function upload($fileData) {
        // Cek jika tidak ada file yang dipilih (error code 4)
        if ($fileData['error'] === 4) {
            return ['status' => 'error', 'msg' => 'Pilih file terlebih dahulu!'];
        }

        $fileName = $fileData['name'];
        $fileSize = $fileData['size'];
        $tmpName = $fileData['tmp_name'];

        $ekstensiFile = strtolower(pathinfo($fileName, PATHINFO_EXTENSION));

        // Validasi ekstensi
        if (!in_array($ekstensiFile, self::$allowedExtensions)) {
            return ['status' => 'error', 'msg' => 'Format file tidak didukung! (Hanya JPG, PNG, PDF, DOCX)'];
        }
    }
}
```

```

    }

    // Validasi ukuran (Maks 2MB)
    if ($fileSize > 2000000) {
        return ['status' => 'error', 'msg' => 'Ukuran file terlalu
besar! (Maksimal 2MB)'];
    }

    // Generate nama unik
    $namaFileBaru = uniqid() . '.' . $ekstensiFile;

    // Buat folder jika belum ada
    if (!is_dir(self::$uploadPath)) {
        mkdir(self::$uploadPath, 0777, true);
    }

    // Pindahkan file dari temp ke folder tujuan
    if (move_uploaded_file($tmpName, self::$uploadPath .
$namaFileBaru)) {
        return ['status' => 'success', 'filename' =>
$namaFileBaru];
    }

    return ['status' => 'error', 'msg' => 'Gagal mengupload file ke
server.'];
}

/**
 * Menghapus file fisik dari server.
 */
public static function delete($fileName) {
    $filePath = self::$uploadPath . $fileName;
    if (!empty($fileName) && file_exists($filePath)) {
        return unlink($filePath);
    }
    return false;
}

/**
 * Menangani proses pembaruan file (upload baru dan hapus yang
lama).
 */
public static function handleUpdate($fileInput, $oldFileName,
$defaultFile = 'default.png') {
    if ($fileInput['error'] !== 4) {
        $uploadResult = self::upload($fileInput);
        if ($uploadResult['status'] === 'success') {
            // Hapus file lama jika ada dan bukan file default

```

```

        if ($oldFileName !== $defaultFile &&
!empty($oldFileName)) {
            self::delete($oldFileName);
        }
        return ['status' => 'success', 'filename' =>
$uploadResult['filename']];
    } else {
        return ['status' => 'error', 'msg' =>
$uploadResult['msg']];
    }
}
// Jika tidak ada upload baru, gunakan file lama
return ['status' => 'no_file', 'filename' => $oldFileName];
}
}

```

src/User.php

```

<?php

/**
 * Class User
 * Model untuk mengelola data pada tabel 'users'.
 */
class User extends BaseModel {
    protected $table = 'users';

    /**
     * Mencari user berdasarkan alamat email.
     */
    public function findByEmail($email) {
        $result = $this->all(['email' => $email]);
        return (!empty($result)) ? $result[0] : null;
    }

    /**
     * Mendaftarkan user baru dengan hashing password otomatis.
     */
    public function register($name, $email, $password, $role) {
        $hashedPassword = password_hash($password, PASSWORD_DEFAULT);
        return $this->create([
            'name' => $name,
            'email' => $email,
            'password' => $hashedPassword,
            'role' => $role
        ]);
    }
}

```

```
}
```

admin.php

```
<?php
// Memuat file inisialisasi (autoloader & session)
require_once 'config/init.php';

// Memastikan hanya user yang sudah login yang bisa mengakses dashboard
Auth::checkLogin();

/**
 * 1. Flow Pengambilan Data
 * Mengambil seluruh data dokter dari database untuk ditampilkan di
 tabel.
 */
$doctorModel = new Doctor();
$dokter = $doctorModel->all();
?>
```

delete.php

```
<?php
// Memuat file inisialisasi (autoloader & session)
require_once 'config/init.php';

// Proteksi halaman dashboard
Auth::checkLogin();

/**
 * Flow Penghapusan Data (Delete)
 */
if (isset($_GET['id'])) {
    $id = $_GET['id'];

    /**
     * 1. Validasi Data
     * Mengambil data untuk mendapatkan nama file sertifikat agar bisa
 dibersihkan.
     */
    $doctorModel = new Doctor();
    $dokter = $doctorModel->find($id);

    if ($dokter) {
        $fileSertifikat = $dokter['sertifikat'] ?? '';

        /**
```

```

        * 2. Bersihkan File Fisik
        * Menghapus file sertifikat dari folder uploads untuk
menghemat storage.
    */
    if (!empty($fileSertifikat)) {
        FileHelper::delete($fileSertifikat);
    }

    /**
    * 3. Hapus Record Database
    * Setelah file sukses dihapus, hapus data dari tabel.
    */
    if ($doctorModel->delete($id)) {
        header("Location: admin.php?deleted=success");
        exit;
    } else {
        header("Location: admin.php?deleted=error");
        exit;
    }
} else {
    header("Location: admin.php"); // ID tidak ditemukan
    exit;
}
} else {
    header("Location: admin.php"); // ID tidak disediakan
    exit;
}
?>

```

edit.php

```

<?php
// Memuat file inisialisasi (autoloader & session)
require_once 'config/init.php';

// Proteksi halaman dashboard
Auth::checkLogin();

/**
* 1. Flow Validasi & Pengambilan Data
* Mengecek apakah ID valid dan datanya tersedia di database.
*/
if (!isset($_GET['id']) || empty($_GET['id'])) {
    header("Location: admin.php");
    exit;
}

```

```

$id = $_GET['id'];
$doctorModel = new Doctor();
$soldData = $doctorModel->find($id);

if (!$soldData) {
    // Jika data tidak ditemukan, redirect kembali
    echo "<script>alert('Data dokter tidak ditemukan!');
window.location='admin.php';</script>";
    exit;
}

/**
 * 2. Flow Pembaruan Data (Update)
 * Menangani pengiriman form edit.
 */
if (isset($_POST['update'])) {
    $data = [
        'nama' => $_POST['nama'],
        'spesialisasi' => $_POST['spesialisasi'],
        'email' => $_POST['email'],
        'telepon' => $_POST['telepon'],
        'nomor_sip' => $_POST['nomor_sip'],
        'kode_dokter' => $_POST['kode_dokter'],
        'bio' => $_POST['bio']
    ];

    /**
     * Handle Update File Sertifikat
     * Jika ada file baru, hapus yang lama dan simpan yang baru.
     */
    $fileUpdate = FileHelper::handleUpdate($_FILES['sertifikat'],
    $soldData['sertifikat']);

    if ($fileUpdate['status'] !== 'error') {
        $data['sertifikat'] = $fileUpdate['filename'];

        try {
            // 3. Update Database: Eksekusi perubahan ke DB
            $doctorModel->update($id, $data);
            $success_msg = "Data dokter berhasil diperbarui!";

            // Update variabel local agar form merefleksikan perubahan
            terbaru
            $soldData = array_merge($soldData, $data);
        } catch (Exception $e) {
            $error_msg = "Gagal memperbarui data!";
        }
    } else {

```

```

        $error_msg = "Gagal update sertifikat: " . $fileUpdate['msg'];
    }
}
?>

```

login.php

```

<?php
// Memuat file inisialisasi (autoloader & session)
require_once 'config/init.php';

// Proteksi halaman: jika sudah login, lempar ke dashboard
Auth::redirectIfLoggedIn();

$error = '';

// Flow Login: Menangani pengiriman form POST
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $email = $_POST['email'];
    $password = $_POST['password'];

    // Mencoba login menggunakan class Auth
    if (Auth::login($email, $password)) {
        header("Location: admin.php");
        exit();
    } else {
        // Jika gagal, cek alasan kegagalan untuk feedback user
        $userModel = new User();
        if (!$userModel->findByEmail($email)) {
            $error = "Email tidak ditemukan!";
        } else {
            $error = "Password salah!";
        }
    }
}
}
?>

```

register.php

```

<?php
// Memuat file inisialisasi (autoloader & session)
require_once 'config/init.php';

// Proteksi halaman: jika sudah login, lempar ke dashboard
Auth::redirectIfLoggedIn();

$error = '';
$success = '';

```

```

// Flow Registrasi: Menangani pengiriman form POST
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $name = $_POST['name'];
    $email = $_POST['email'];
    $password = $_POST['password'];
    $confirm_password = $_POST['confirm_password'];
    $role = $_POST['role'];

    $userModel = new User();

    // 1. Validasi Email: Cek apakah email sudah terdaftar
    $check_email = $userModel->findByEmail($email);

    if ($check_email) {
        $error = "Email sudah terdaftar!";
    } elseif ($password !== $confirm_password) {
        $error = "Password dan Konfirmasi Password tidak cocok!";
    } else {
        // 2. Flow Khusus Dokter: Menangani upload sertifikat
        $doctorData = null;
        if ($role === 'doctor') {
            if (isset($_FILES['cert_file']) &&
$_FILES['cert_file']['error'] === 0) {
                $upload = FileHelper::upload($_FILES["cert_file"]);
                if ($upload['status'] === 'success') {
                    $doctorData = [
                        "nama" => $name,
                        "sertifikat" => $upload["filename"],
                        "email" => $email
                    ];
                } else {
                    $error = $upload['msg'];
                }
            } else {
                $error = "File sertifikasi (STR/SIP) wajib diupload
untuk dokter!";
            }
        }

        // 3. Simpan Data: Jika validasi lolos, masukkan ke database
        if (empty($error)) {
            if ($userModel->register($name, $email, $password, $role))
            {
                // Jika role adalah dokter, simpan juga ke tabel dokter
                if ($role === 'doctor' && $doctorData) {
                    $doctorModel = new Doctor();
                    $doctorModel->create($doctorData);
                }
            }
        }
    }
}

```

```

        }
        $success = "Registrasi berhasil! Silakan login.";
    } else {
        $error = "Terjadi kesalahan saat registrasi.";
    }
}
}
}
?>

```

tambah-dokter.php

```

<?php
// Memuat file inisialisasi (autoloader & session)
require_once 'config/init.php';

// Proteksi halaman dashboard
Auth::checkLogin();

/**
 * Flow Penambahan Dokter Baru (Create)
 * Dijalankan ketika form disubmit (tombol 'tambah' diklik).
 */
if (isset($_POST['tambah'])) {
    $dokumen = $_FILES['sertifikat'] ?? null;

    // 1. Upload File: Memanggil helper FileHelper::upload untuk validasi
    // dan simpan file
    $uploadResult = FileHelper::upload($dokumen);

    if ($uploadResult['status'] == "success") {
        $data = $_POST;
        unset($data['tambah']); // Membersihkan key 'tambah' dari array
        // POST
        $data['sertifikat'] = $uploadResult['filename']; // Mengaitkan nama
        // file yang baru diupload

        $doctorModel = new Doctor();
        try {
            // 2. Simpan Database: Gunakan model untuk menyisipkan data
            // dokter baru
            $doctorModel->create($data);
            $success_msg = "Dokter berhasil ditambahkan!";
        } catch (Exception $e) {
            $error_msg = "Error! pastikan email dan kode dokter belum
            terdaftar!";
        }
    } else {

```

```
// Feedback jika upload gagal
$error_msg = "Gagal upload sertifikat: " . $uploadResult['msg'];
}
}
?>
```

b. Dokumentasi

Hi! Selamat Datang di Nuids.



Nama Lengkap

Email

Password

Konfirmasi Password

Pilih tipe akun:

☒ Pengguna ☐ Dokter

Register

Sudah punya akun? [Login](#)

Hi! Selamat Datang Kembali di Nuids.



Email

Password

Login

Belum punya akun? [Daftar sekarang](#)

Manajemen Dokter



admin
Super Administrator

Manajemen Dokter

Daftar Dokter

+ Tambah Dokter

Cari dokter...

Semua Spesialisasi

Nama Dokter	Spesialisasi	Kontak	Sertifikat	Aksi
Andhika Yuridho	Dokter Anak	andhikayuridho05@gmail.com 081879328163	Lihat File	DetailEditHapus

Kembali ke daftar dokter

Form Data Dokter

Tambah Dokter Baru

Lengkapi data dokter di bawah ini untuk menambahkan ke dalam sistem Nuids.

01Informasi Utama

Data identitas yang akan tampil paling sering di daftar dokter.

Nama Dokter *

Andhika Yuridho

Gunakan nama lengkap sesuai format tampilan profil dokter.

Spesialisasi *

Spesialis Anak

Pilih yang paling mendekati kategori layanan dokter.

Email *

andhikayuridho05@gmail.com

Dipakai untuk kebutuhan administrasi dan komunikasi internal.

Nomor Telepon *

081879328163

Masukkan nomor yang aktif dan mudah dihubungi.

02Profil Tambahan

Lengkapi informasi pendukung untuk administrasi dan unggah dokumen pendukung.

Nomor SIP / STR

441/428/Pelayanan dan sumber daya kes/20

Simpan nomor izin agar mudah diverifikasi saat dibutuhkan.

Kode / ID Dokter

ID_123

Opsional untuk kebutuhan penandaan internal atau integrasi sistem.

Deskripsi Singkat

"Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum

Ringkas saja, cukup 1-3 kalimat agar tetap mudah dibaca.

Unggah Sertifikat / Dokumen *

Choose File

1361020.png

Format yang didukung: PDF, DOCX, JPG, atau PNG (Maks. 5MB).

Batal

Tambah Dokter

E. Analisis dan Pembahasan

Refaktorisasi aplikasi dari model pemrograman prosedural ke Object-Oriented Programming (OOP) membawa perubahan fundamental dalam cara aplikasi mengelola data dan alur kerja. Dalam struktur prosedural sebelumnya, fungsi-fungsi yang bersifat umum tercampur dalam satu berkas, yang mengakibatkan tingginya ketergantungan antar bagian (tight coupling) dan menyulitkan proses debugging saat aplikasi berkembang. Dengan menerapkan konsep kelas dan objek, logika program kini terfragmentasi menjadi unit-unit mandiri yang memiliki tanggung jawab tunggal (Single Responsibility Principle). Analisis ini menunjukkan bahwa struktur baru mempermudah pengelolaan kode melalui pengorganisasian kelas berdasarkan entitas, seperti kelas User untuk autentikasi dan kelas Doctor untuk pengelolaan data medis, sehingga meminimalisir redundansi kode secara signifikan.

Penerapan kelas abstrak BaseModel merupakan poin krusial dalam pembahasan ini. Metode CRUD (Create, Read, Update, Delete) yang bersifat generik dipusatkan pada satu tempat, sehingga model entitas lainnya hanya perlu mewarisi fungsi tersebut tanpa harus menulis ulang query SQL yang kompleks. Hal ini tidak hanya mempercepat waktu pengembangan, tetapi juga meningkatkan konsistensi data karena setiap operasi database kini melalui satu pintu yang sama. Selain itu, penggunaan pola desain Singleton pada kelas Database memastikan pemakaian memori yang lebih efisien dengan menghindari pembuatan koneksi database berulang kali dalam satu siklus eksekusi program. Sistem ini memberikan fondasi yang kuat bagi aplikasi untuk dikembangkan lebih lanjut (scalability) di masa depan.

Dari sisi keamanan, transisi ke OOP ini juga mengintegrasikan penggunaan Prepared Statements secara otomatis di setiap metode database pada BaseModel. Analisis menunjukkan bahwa pendekatan ini secara efektif menutup celah kerentanan terhadap serangan SQL Injection, karena nilai input dari pengguna dipisahkan dari struktur kueri melalui proses binding parameter. Selain itu, pemisahan logika manajemen sesi ke dalam kelas Auth dan utilitas file ke dalam FileHelper membuat alur program menjadi lebih deskriptif dan terlindungi. Secara keseluruhan, pembahasan ini menyimpulkan bahwa meskipun struktur OOP membutuhkan perencanaan awal yang lebih kompleks, hasil akhirnya memberikan kode yang jauh lebih bersih, aman, dan mudah untuk dilakukan pemeliharaan jangka panjang (long-term maintenance).

F. Kesimpulan

Berdasarkan hasil refaktorisasi yang telah dilakukan, dapat disimpulkan bahwa penerapan konsep Object-Oriented Programming (OOP) pada aplikasi manajemen dokter ini berhasil meningkatkan kualitas kode dari segi struktur, keamanan, maupun efisiensi. Transisi dari paradigma prosedural ke OOP memungkinkan aplikasi untuk memiliki kode yang lebih modular melalui pengorganisasian kelas-kelas yang saling independen namun tetap terintegrasi secara harmonis. Penggunaan pola desain Singleton pada koneksi database dan penerapan Inheritance (pewarisan) melalui BaseModel terbukti dapat meminimalisir redundansi kode serta mengoptimalkan penggunaan sumber daya server.

Selain itu, aspek keamanan aplikasi mengalami peningkatan signifikan dengan penggunaan Prepared Statements yang terstandarisasi di dalam kelas model. Implementasi Autoloading dan sistem inisialisasi yang terpusat juga membuat kode menjadi jauh lebih bersih dan mempermudah pengembang dalam melakukan pemeliharaan maupun pengembangan fitur baru di masa mendatang. Secara keseluruhan, penerapan OOP bukan sekadar perubahan gaya penulisan, melainkan sebuah transformasi arsitektur yang membuat aplikasi menjadi lebih profesional, aman, dan siap untuk dikembangkan ke skala yang lebih luas..

G. Link Repository Github

<https://github.com/AriqAhmadNayaka/WebPro4904/tree/AndhikaYuridho-707012500059>